B Bidding System - AI Pilot Implementation Plan

Rufus SDK + FastAPI Architecture

Document Version: 1.0 **Date**: 2026-02-11 **Status**: Planning Phase

---

Executive Summary

This document outlines the implementation plan for the **B Bidding Master** system - an AI-powered RFP intake, bid decisioning, and sub-consultant management platform built on **Rufus SDK** with **FastAPI**. This represents a production-ready pilot demonstrating advanced AI orchestration for architecture/engineering firms.

Strategic Value

- **Proven AI Use Case**: End-to-end automation of complex, multi-party RFP workflows
- **Time Savings**: 60-80% reduction in manual RFP triage and sub-consultant coordination
- **Scalability**: Distributed microservices architecture handling concurrent RFPs
- **Auditability**: Full workflow history for compliance and continuous improvement

Technology Stack

- **Workflow Engine**: Rufus SDK (Python-native, production-tested)
- **API Layer**: FastAPI (async, OpenAPI, high performance)
- **Execution**: Celery + Redis (distributed task queue)
- **Persistence**: PostgreSQL (JSONB, full-text search, audit logs)
- **AI/ML**: OpenAI/Anthropic APIs, LangChain for document processing
    - **Phase 1**: LangChain + chunking (standard approach)
    - **Phase 2+ (Optional)**: Recursive Language Models (RLM) for complex multi-document analysis
- **Storage**: S3-compatible (MinIO for dev, AWS S3 for production)
- **Frontend**: React SPA with TypeScript, Tailwind CSS, shadcn/ui

---

Table of Contents

1. System Architecture
2. Custom Step Types (Marketplace Extension)
    - 2.4 Advanced Enhancement: Recursive Language Models (RLM)
3. Core Workflows
4. FastAPI API Design
5. Frontend Architecture
6. Database Schema
7. Deployment Architecture
8. Implementation Phases
9. Risk Mitigation
10. Success Metrics

---

1. System Architecture

1.1 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────────────
│                          FRONTEND (React SPA)
│   ┌──────────────┬──────────────┬──────────────┬──────────────
│   │  RFP Intake  │   Dashboards | Partner      |  Contract
│   │  Portal      |   Analytics  | Portal       |  Review
│   └──────────────┴──────────────┴──────────────┴──────────────
└──────────────────────────────────────────────────────────────────────────────
                                    | HTTPS/REST API
┌──────────────────────────────────────────────────────────────────────────────
│                        FASTAPI API GATEWAY
│   ┌──────────────────────────────────────────────────────────────────────
│   │  Authentication (JWT) | Authorization | Rate Limiting
│   └──────────────────────────────────────────────────────────────────────
│   ┌──────────────────────────────────────────────────────────────────────
│   │  /api/v1/rfps        | /api/v1/workflows  | /api/v1/partr
│   │  /api/v1/contracts   | /api/v1/analytics  | /api/v1/clier
│   └──────────────────────────────────────────────────────────────────────
└──────────────────────────────────────────────────────────────────────────────
                                    |
┌──────────────────────────────────────────────────────────────────────────────
│                      RUFUS WORKFLOW ENGINE
│   ┌──────────────────────────────────────────────────────────────────────
│   │  WorkflowBuilder  |  Workflow  |  ExecutionProvider
│   └──────────────────────────────────────────────────────────────────────
│   ┌──────────────────────────────────────────────────────────────────────
│   │  Custom Step Types (rufus-bidding-steps):
│   │  • AI_EXTRACTION      • AI_SCORING      • AI_CONTRACT_REV
│   │  • FILE_PROCESSOR     • EMAIL_NOTIFIER  • TEMPLATE_GENERA
│   └──────────────────────────────────────────────────────────────────────
└──────────────────────────────────────────────────────────────────────────────
         |                  |                    |
     ┌──────────┐       ┌──────────┐        ┌──────────┐
     | Celery   |       | Redis    |        |  S3      |
     | Workers  |◄──────|  Queue   |        | Storage  |
     |  (5-10)  |       |          |        |  (RFPs)  |
     └──────────┘       └──────────┘        └──────────┘
         |
     ┌──────────────────────────────┐
     |  PostgreSQL (Primary)        |
     |  • workflow_executions       |
     |  • rfps (full-text index)    |
     |  • partners (PQQ data)       |
     |  • contracts (clauses)       |
     |  • analytics_metrics         |
     └──────────────────────────────┘
```

1.2 Microservices Breakdown

Service	Technology	Responsibility	Scaling

API Gateway	FastAPI	REST endpoints, auth, request routing	Horizontal (3+ instances)
Workflow Manager	Rufus SDK + FastAPI	Workflow orchestration, state management	Stateless (3+ instances)
Celery Workers	Celery + Python	Async task execution (AI, file processing)	Horizontal (5-10 workers)
Document Processor	Python + LangChain	PDF/Excel/Word extraction, OCR	Dedicated worker pool (2-3)
AI Service	OpenAI/Anthropic	LLM-based extraction, scoring, analysis	API calls (rate-limited)
Notification Service	Celery + SMTP/SendGrid	Email reminders to sub-consultants	Shared worker pool
Analytics Engine	PostgreSQL + Metabase	BI dashboards, win/loss analysis	Read replicas

1.3 Data Flow Example: RFP Intake to Bid Decision

```
1. User uploads RFP ZIP file (10 GB)
   |
   ├─► FastAPI: POST /api/v1/rfps/upload
   |    ├─► S3: Store ZIP file (rfps/2026-02-11/rfp_12345.zip)
   |    └─► Rufus: Start "RFP_Intake" workflow
   |
2. Workflow Step: FILE_PROCESSOR (ASYNC, Celery worker)
   |    ├─► Unzip archive (100+ files)
   |    ├─► Extract metadata (filenames, sizes, types)
   |    └─► Return: {"files": [{name, type, size, s3_key}]}
   |
3. Workflow Step: AI_EXTRACTION (ASYNC, Celery worker)
   |    ├─► LangChain: Load PDFs with chunking
   |    ├─► OpenAI: Extract key fields (GFA, BUA, location, de
   |    └─► Return: {"project_name": "...", "gfa_sqm": 5000, .
   |
4. Workflow Step: AI_SCORING (STANDARD, immediate)
   |    ├─► Load Bronwyn's scoring model (A/B/C/D → points)
   |    ├─► Apply rules: location, project size, client histor
   |    └─► Return: {"score": 75, "recommendation": "BID", "re
   |
5. Workflow Step: HUMAN_REVIEW (PAUSE)
   |    ├─► Status: WAITING_HUMAN
   |    ├─► Notification: Email to Bronwyn (approval link)
   |    └─► Dashboard: Shows pending decision
   |
6. User approves/overrides decision
   |
   ├─► FastAPI: POST /api/v1/workflows/{id}/resume
   |    └─► Rufus: Resume workflow with {"approved": true, "ov
   |
7. Workflow Step: DECISION (immediate)
   |    ├─► If approved: Jump to "INVITE_CONSULTANTS"
   |    └─► If declined: Jump to "GENERATE_DECLINE_EMAIL"
   |
8. Sub-Workflow: "CONSULTANT_INVITATION" (30 consultants)
   |    ├─► Create RFP package per consultant
   |    ├─► Send email invitations with portal links
   |    └─► Register CRON job: reminder in 2 days
```

---

2. Custom Step Types (Marketplace Extension)

To support the B Bidding domain, we'll create **`rufus-bidding-steps`** as a separate Python package with custom step types.

2.1 Package Structure

```
rufus-bidding-steps/
├── pyproject.toml  # Entry points for Rufus discovery
├── src/
|   └── rufus_bidding/
|       ├── __init__.py
|       ├── models.py           # Custom step type definitions
|       ├── steps/
|       |   ├── ai_extraction.py
|       |   ├── ai_scoring.py
|       |   ├── ai_contract_review.py
|       |   ├── file_processor.py
|       |   ├── email_notifier.py
|       |   └── template_generator.py
|       ├── providers/
|       |   ├── llm_provider.py    # OpenAI/Anthropic abstract
|       |   └── storage_provider.py # S3/MinIO abstraction
|       └── utils/
|           ├── pdf_parser.py
|           ├── excel_parser.py
|           └── scoring_engine.py
└── tests/
```

2.2 Custom Step Type Definitions

AI_EXTRACTION Step

```python
# src/rufus_bidding/models.py
from rufus.models import WorkflowStep
from pydantic import BaseModel, Field
from typing import List, Dict, Optional

class ExtractionField(BaseModel):
    """Defines a field to extract from documents."""
    name: str
    description: str
    data_type: str = "string"  # string, number, date, list, b
    required: bool = True
    validation_regex: Optional[str] = None

class AIExtractionStep(WorkflowStep):
    """
    Extracts structured data from unstructured documents using

    Example YAML:
    - name: "Extract_RFP_Data"
      type: "AI_EXTRACTION"
      extraction_config:
        document_keys: ["main_rfp_pdf", "technical_brief_pdf"]
        fields:
          - name: "project_name"
            description: "Full project name"
            data_type: "string"
            required: true
          - name: "gfa_sqm"
            description: "Gross Floor Area in square meters"
            data_type: "number"
            required: true
        llm_provider: "openai"
        model: "gpt-4o"
        temperature: 0.1
        chunk_size: 2000
        max_retries: 3
    """
    type: str = Field("AI_EXTRACTION", const=True)
    extraction_config: Dict[str, Any] = Field(
        ...,
        description="Configuration for AI extraction"
    )

    # Fields from extraction_config (validated by builder)
    document_keys: List[str]  # Keys in state containing S3 pa
    fields: List[ExtractionField]  # Fields to extract
    llm_provider: str = "openai"  # openai, anthropic
    model: str = "gpt-4o"
    temperature: float = 0.1
    chunk_size: int = 2000
    max_retries: int = 3
    timeout_seconds: int = 300
```

Execution Logic (in `Workflow.next_step()`):

```
elif isinstance(step, AIExtractionStep):
    # Dispatch to Celery worker (long-running)
    result = await self.execution_provider.dispatch_async_task
        func_path="rufus_bidding.steps.ai_extraction.execute",
        state_data=self.state.model_dump(),
        workflow_id=self.id,
        context_data=context.model_dump(),
        step_config=step.extraction_config,
        timeout=step.timeout_seconds
    )
    # Worker will call LLM, parse documents, return extracted
    self._apply_merge_strategy(self.state, result, step.merge_
```

AI_SCORING Step

```python
class ScoringRule(BaseModel):
    """Defines a scoring rule."""
    name: str
    field: str  # Field in state to evaluate
    rule_type: str  # range, lookup, boolean, custom
    parameters: Dict[str, Any]
    points: int  # Points awarded if rule matches

class AIScoringStep(WorkflowStep):
    """
    Applies scoring model with optional AI enhancement.

    Example YAML:
    - name: "Score_RFP"
      type: "AI_SCORING"
      scoring_config:
        rules:
          - name: "Project Size"
            field: "gfa_sqm"
            rule_type: "range"
            parameters:
              ranges:
                - {min: 0, max: 1000, points: 10}
                - {min: 1001, max: 5000, points: 20}
                - {min: 5001, max: 10000, points: 30}
            points: 0  # Overridden by range
          - name: "Client History"
            field: "client_name"
            rule_type: "lookup"
            parameters:
              lookup_table: "state.client_ratings"
            points: 0
        ai_enhancement:
          enabled: true
          model: "gpt-4o"
          prompt: "Analyze project complexity and strategic va
        recommendation_thresholds:
          A: 80  # BID (high priority)
          B: 60  # BID (conditional)
          C: 40  # NO-BID (with explanation)
          D: 0   # NO-BID (clear)
    """
    type: str = Field("AI_SCORING", const=True)
    scoring_config: Dict[str, Any]

    rules: List[ScoringRule]
    ai_enhancement: Optional[Dict[str, Any]] = None
    recommendation_thresholds: Dict[str, int]
```

AI_CONTRACT_REVIEW Step

```python
class ContractClause(BaseModel):
    """Defines a clause to flag in contracts."""
    name: str
    keywords: List[str]
    risk_level: str  # LOW, MEDIUM, HIGH, CRITICAL
    description: str
    whitebook_reference: Optional[str] = None

class AIContractReviewStep(WorkflowStep):
    """
    Analyzes contract documents for risky clauses.

    Example YAML:
    - name: "Review_Contract"
      type: "AI_CONTRACT_REVIEW"
      contract_config:
        document_keys: ["contract_pdf"]
        clauses_to_flag:
          - name: "Performance Bond"
            keywords: ["performance bond", "bank guarantee"]
            risk_level: "HIGH"
            description: "Financial guarantee required"
            whitebook_reference: "Section 4.2"
        whitebook_path: "state.whitebook_s3_key"
        comparison_enabled: true
        llm_provider: "anthropic"
        model: "claude-3-5-sonnet-20241022"
    """
    type: str = Field("AI_CONTRACT_REVIEW", const=True)
    contract_config: Dict[str, Any]

    document_keys: List[str]
    clauses_to_flag: List[ContractClause]
    whitebook_path: Optional[str] = None
    comparison_enabled: bool = True
    llm_provider: str = "anthropic"
    model: str = "claude-3-5-sonnet-20241022"
```

FILE_PROCESSOR Step

```python
class FileProcessorStep(WorkflowStep):
    """
    Processes uploaded files (unzip, validate, extract metadat

    Example YAML:
    - name: "Process_RFP_Package"
      type: "FILE_PROCESSOR"
      file_config:
        input_key: "uploaded_file_s3_key"
        operations:
          - type: "unzip"
            parameters:
              max_files: 1000
              exclude_patterns: ["__MACOSX", ".DS_Store"]
          - type: "validate"
            parameters:
              allowed_extensions: [".pdf", ".docx", ".xlsx", "
              max_file_size_mb: 100
          - type: "extract_metadata"
            parameters:
              include_checksums: true
        output_key: "processed_files"
        storage_path: "rfps/{{workflow_id}}/files/"
    """
    type: str = Field("FILE_PROCESSOR", const=True)
    file_config: Dict[str, Any]

    input_key: str
    operations: List[Dict[str, Any]]
    output_key: str = "processed_files"
    storage_path: str
    timeout_seconds: int = 600  # 10 min for large zips
```

EMAIL_NOTIFIER Step

```python
class EmailTemplate(BaseModel):
    """Email template with Jinja2 variables."""
    subject: str
    body: str  # Jinja2 template
    recipients: List[str]  # Can be state paths: "{{state.cons
    cc: Optional[List[str]] = None
    attachments: Optional[List[str]] = None  # S3 keys

class EmailNotifierStep(WorkflowStep):
    """
    Sends emails with template rendering.

    Example YAML:
    - name: "Invite_Consultants"
      type: "EMAIL_NOTIFIER"
      email_config:
        template:
          subject: "RFP Invitation: {{state.project_name}}"
          body: |
            Dear {{state.consultant_name}},

            You are invited to bid on: {{state.project_name}}
            Deadline: {{state.submission_deadline}}

            Portal link: {{state.portal_url}}
          recipients: ["{{state.consultant_email}}"]
          attachments: ["{{state.rfp_package_s3_key}}"]
        smtp_config:
          provider: "sendgrid"  # sendgrid, smtp
          api_key: "{{env.SENDGRID_API_KEY}}"
        retry_policy:
          max_retries: 3
          backoff_seconds: 60
    """
    type: str = Field("EMAIL_NOTIFIER", const=True)
    email_config: Dict[str, Any]

    template: EmailTemplate
    smtp_config: Dict[str, Any]
    retry_policy: Optional[Dict[str, Any]] = None
```

TEMPLATE_GENERATOR Step

```python
class TemplateGeneratorStep(WorkflowStep):
    """
    Auto-populates Word/Excel templates with workflow data.

    Example YAML:
    - name: "Generate_Technical_Proposal"
      type: "TEMPLATE_GENERATOR"
      template_config:
        template_s3_key: "templates/technical_proposal_v2.docx
        output_filename: "Technical_Proposal_{{state.project_r
        data_mappings:
          "{{PROJECT_NAME}}": "state.project_name"
          "{{GFA}}": "state.gfa_sqm"
          "{{TEAM_MEMBERS}}": "state.team_members"  # List
        format: "docx"  # docx, xlsx, pptx
        output_key: "technical_proposal_s3_key"
    """
    type: str = Field("TEMPLATE_GENERATOR", const=True)
    template_config: Dict[str, Any]

    template_s3_key: str
    output_filename: str
    data_mappings: Dict[str, str]
    format: str = "docx"
    output_key: str = "generated_document_s3_key"
```

2.3 Marketplace Registration

pyproject.toml:

```toml
[project.entry-points."rufus.steps"]
ai_extraction = "rufus_bidding.models:AIExtractionStep"
ai_scoring = "rufus_bidding.models:AIScoringStep"
ai_contract_review = "rufus_bidding.models:AIContractReviewSte
file_processor = "rufus_bidding.models:FileProcessorStep"
email_notifier = "rufus_bidding.models:EmailNotifierStep"
template_generator = "rufus_bidding.models:TemplateGeneratorSt

[project.entry-points."rufus.providers"]
llm_provider = "rufus_bidding.providers:LLMProvider"
storage_provider = "rufus_bidding.providers:StorageProvider"
```

Installation:

```
# Development
pip install -e rufus-bidding-steps/

# Production
pip install rufus-bidding-steps
```

Rufus Auto-Discovery: WorkflowBuilder automatically loads entry points on initialization.

2.4 Advanced Enhancement: Recursive Language Models (RLM)

Status: Phase 2+ Optional Enhancement (Not Required for Phase 1)

What are RLMs?

Recursive Language Models (RLMs), introduced by Zhang et al. (2025), represent a paradigm shift in handling documents exceeding LLM context windows. Instead of "stuffing" massive prompts into the model's neural network, RLMs treat long documents as external environment variables manipulated via Python code.

Key Innovation: The LLM doesn't "read" the entire document. Instead, it acts as a coordinator (Root Model) that writes Python code to search, slice, and recursively analyze document sections.

How RLMs Work

1. **Environment Setup**: User prompt stored as variable (e.g., `context = rfp_document_text`)
2. **Code Generation**: Root LLM generates Python to interact with context (e.g., `re.finditer("performance bond", context)`)
3. **Recursive Sub-calls**: Model calls `llm_query(sub_prompt, context_slice)` for specific sections
4. **Stateful Reasoning**: Updates `answer` dict with `ready: True` when complete

Example RLM Workflow:

```python
# Root Model generates this code
import re

# Find all mentions of "liquidated damages" across all documer
ld_mentions = []
for doc_key in state.document_keys:
    doc_text = load_document(doc_key)
    matches = re.finditer(r"liquidated damages?.*?\d+%", doc_t

    for match in matches:
        # Recursive sub-query to analyze each mention
        context_slice = doc_text[match.start()-500:match.end()
        analysis = llm_query(
            prompt="Analyze this liquidated damages clause for
            context=context_slice
        )
        ld_mentions.append(analysis)

answer = {
    "content": {"ld_clauses": ld_mentions, "total_risk": "HIGH
    "ready": True
}
```

When RLMs Outperform Standard Chunking

Scenario	Standard Chunking	RLM Advantage	B Bidding Impact
Cross-document synthesis	5 sequential LLM calls, $0.50 cost	1 RLM call, $0.30 cost (40% savings)	Better risk summaries
Contract vs. White Book	Manual section mapping required	Automatic cross-referencing	3-5x faster analysis
Mega-RFPs (50+ MB)	Token cost: $2-5 per RFP	Token cost: $1-2 per RFP (50% savings)	Feasible for large projects
Subtle contradictions	Often missed across chunks	Recursive structure finds conflicts	Higher accuracy (+10-20%)
Complex queries	"Find X, then find Y based on X" requires multiple calls	Single RLM call handles logic	Simplified workflow

Research Results (Zhang et al., 2025):

- **Context Scaling**: Successfully processed 10 million tokens (100x beyond frontier models)
- **Accuracy**: Maintained near-perfect retrieval on "Needle In A Haystack" tests at 10M tokens
- **Cost**: GPT-5-mini + RLM processed 10M tokens for $0.99 (vs. $5+ with standard long-context)
- **Performance**: RLM-Qwen3-8B outperformed base model by 28.3% on long-context benchmarks

RLM_QUERY Step Type (Phase 2+ Implementation)

Step Definition:

```python
# In rufus-bidding-steps/src/rufus_bidding/models.py

class RLMConfig(BaseModel):
    """Configuration for RLM-based multi-document analysis."""

    # Documents to analyze
    document_keys: List[str]  # State paths to S3 URLs

    # LLM configuration
    llm_provider: str = "anthropic"  # For Claude's prompt cac
    model: str = "claude-opus-4-6"
    temperature: float = 0.2
    enable_prompt_caching: bool = True  # Reuse document embec

    # Query specification
    query: str  # Question to answer across documents
    extraction_schema: Optional[Dict[str, Any]] = None  # JSON

    # Execution limits
    timeout_seconds: int = 600  # 10 min for complex analysis
    max_recursion_depth: Optional[int] = None  # Auto-optimal

class RLMQueryStep(WorkflowStep):
    """
    RLM-based multi-document analysis step.

    Example YAML:
    - name: "Comprehensive_Risk_Analysis"
      type: "RLM_QUERY"
      rlm_config:
        document_keys:
          - "state.rfp_contract_s3_key"
          - "state.technical_spec_s3_key"
          - "state.whitebook_s3_key"
        query: |
          Analyze all documents together:
          1. Identify contractual risks against white book stc
          2. Flag all insurance/bond requirements
          3. Assess schedule risk factors
          Return structured JSON with risk levels and recommer
        extraction_schema:
          type: "object"
          properties:
            risks: {type: "array"}
            insurance_required: {type: "boolean"}
            total_risk_level: {type: "string", enum: ["LOW", "
        enable_prompt_caching: true
        timeout_seconds: 300
      automate_next: true
    """
    type: str = Field("RLM_QUERY", const=True)
    rlm_config: RLMConfig
```

```
    merge_strategy: MergeStrategy = MergeStrategy.SHALLOW
```

Integration with Rufus SDK

Execution Handler (in `Workflow.next_step()`):

```
elif isinstance(step, RLMQueryStep):
    # Dispatch to Celery worker (long-running recursive analys
    result = await self.execution_provider.dispatch_async_task
        func_path="rufus_bidding.steps.rlm_query.execute",
        state_data=self.state.model_dump(),
        workflow_id=self.id,
        rlm_config=step.rlm_config.model_dump(),
        timeout=step.rlm_config.timeout_seconds
    )

    # Error fallback: If RLM fails, retry with standard chunki
    if result.get("rlm_error"):
        logger.warning(f"RLM failed, falling back to chunking:
        result = await fallback_to_chunking(step.rlm_config.dc

    self._apply_merge_strategy(self.state, result, step.merge_
```

Sandbox Implementation: Leverage existing `JavaScriptWorkflowStep` pattern with Python sandbox (similar memory limits, timeouts).

Hybrid Strategy: When to Use RLM vs. LangChain

Use Standard LangChain + Chunking (Phase 1, Default):

- ✅ Standard RFP extraction (project name, GFA, deadlines)
- ✅ Simple clause detection ("Does contract mention insurance?")
- ✅ Per-document analysis (independent documents)
- ✅ Cost: $0.05-0.20/RFP, Latency: 30-60 sec
- ✅ **Use for: 95% of B Bidding workflows**

Use RLM_QUERY (Phase 2+, Selective):

- 🎯 Cross-document risk synthesis ("Total exposure across all clauses?")
- 🎯 Contract compliance checking (RFP vs. White Book comparison)
- 🎯 Conditional analysis ("If performance bond required, what's schedule impact?")
- 🎯 Large RFP packages (> 50 MB, 50+ pages)
- 🎯 Cost: 30-50% cheaper for multi-document, Latency: 60-120 sec
- 🎯 **Use for: 5-10% of high-value contracts (>$1M)**

Implementation Roadmap

Phase 1 (Immediate): LangChain Only ✅

- Implement AI_EXTRACTION, AI_SCORING, AI_CONTRACT_REVIEW with standard chunking
- Deploy with 100+ test RFPs
- Metrics: extraction accuracy, cost/RFP, processing time
- **Timeline**: 2-3 weeks

Phase 2 (After Phase 1 Validation): RLM Pilot 🆕

- Implement RLM_QUERY step type (~22 hours development)
- A/B test on 20 high-value contracts (>$1M)
- Measure: cost savings, accuracy improvement, latency
- **Go/No-Go Decision**: Proceed if 15%+ accuracy gain OR 25%+ cost savings
- **Timeline**: 3-4 weeks

Phase 3 (Optional, Data-Driven): RLM as Default for Complex Analysis 🔮

- If Phase 2 validates ROI, make RLM default for multi-document workflows
- Keep LangChain for simple extraction (still 90% of use cases)
- **Timeline**: 2-3 weeks

Risk Mitigation

Risk	Mitigation
RLM code generation errors	Automatic fallback to LangChain chunking on timeout/error
Different results from baseline	A/B test with manual review of discrepancies
API dependency (Anthropic-specific)	Design RLM_QUERY to support OpenAI, Anthropic, local models
Cost overruns	Set token budget per RFP (e.g., max 500K tokens), monitor spending
Debugging complexity	Log all generated Python code + recursive calls for audit

Expected ROI (Phase 2 Metrics to Validate)

Conservative Estimate (based on Zhang et al. 2025 paper):

- **Accuracy Improvement**: +10-15% on cross-document risk detection
- **Cost Savings**: 25-40% for multi-document analysis (RFPs with 5+ documents)

- **Latency**: Similar or 20-30% faster (hierarchical compression avoids sequential chunking)
- **Applicability**: 5-10% of B Bidding workflows (high-value contracts only)

Break-even Analysis:

- RLM development cost: ~22 hours × $150/hr = $3,300
- Cost savings per RFP: $0.20 (conservative)
- Break-even: 16,500 RFPs with 5+ documents analyzed
- If 10% of 1000 RFPs/year qualify → break-even in ~165 years (NOT justified on cost alone)
- **Justification**: Accuracy improvement is primary value driver, not cost savings

Recommendation: Only implement RLM if Phase 2 A/B testing shows **measurable accuracy improvement** (10%+ better risk detection) that increases Bronwyn's decision confidence.

---

3. Core Workflows

3.1 Workflow: RFP_Intake

Purpose: Capture RFP, extract data, recommend bid/no-bid decision.

Workflow YAML (`config/workflows/rfp_intake.yaml`):

```yaml
workflow_type: "RFP_Intake"
workflow_version: "1.0.0"
initial_state_model: "b_bidding.models.RFPIntakeState"
description: "RFP intake with AI extraction and bid recommendc

steps:
  # Step 1: Upload and store RFP package
  - name: "Store_RFP_Package"
    type: "STANDARD"
    function: "b_bidding.steps.store_rfp_package"
    input_schema:
      type: "object"
      properties:
        uploaded_file:
          type: "string"
          description: "Uploaded file (multipart/form-data)"
      required: ["uploaded_file"]
    automate_next: true

  # Step 2: Process uploaded files (unzip, validate)
  - name: "Process_Files"
    type: "FILE_PROCESSOR"
    file_config:
      input_key: "rfp_package_s3_key"
      operations:
        - type: "unzip"
          parameters:
            max_files: 1000
            exclude_patterns: ["__MACOSX", ".DS_Store"]
        - type: "validate"
          parameters:
            allowed_extensions: [".pdf", ".docx", ".xlsx", ".j
            max_file_size_mb: 500
        - type: "extract_metadata"
          parameters:
            include_checksums: true
      output_key: "processed_files"
      storage_path: "rfps/{{workflow_id}}/files/"
    dependencies: ["Store_RFP_Package"]
    automate_next: true
    timeout_seconds: 600

  # Step 3: AI extraction of key RFP fields
  - name: "Extract_RFP_Data"
    type: "AI_EXTRACTION"
    extraction_config:
      document_keys: ["processed_files"]  # All PDFs from prev
      fields:
        - name: "project_name"
          description: "Full project name or title"
          data_type: "string"
          required: true
```

```yaml
      - name: "project_description"
        description: "Brief project description"
        data_type: "string"
        required: true
      - name: "gfa_sqm"
        description: "Gross Floor Area in square meters"
        data_type: "number"
        required: false
      - name: "bua_sqm"
        description: "Built-Up Area in square meters"
        data_type: "number"
        required: false
      - name: "num_buildings"
        description: "Number of buildings"
        data_type: "number"
        required: false
      - name: "plot_size_sqm"
        description: "Plot size in square meters"
        data_type: "number"
        required: false
      - name: "typologies"
        description: "Building typologies (e.g., residential
        data_type: "list"
        required: false
      - name: "client_name"
        description: "Client or organization name"
        data_type: "string"
        required: true
      - name: "location"
        description: "Project location (city, country)"
        data_type: "string"
        required: true
      - name: "submission_deadline"
        description: "RFP submission deadline (ISO8601 date)
        data_type: "date"
        required: true
      - name: "disciplines_required"
        description: "Required engineering disciplines (stru
        data_type: "list"
        required: false
    llm_provider: "openai"
    model: "gpt-4o"
    temperature: 0.1
    chunk_size: 2000
    max_retries: 3
  dependencies: ["Process_Files"]
  automate_next: true
  timeout_seconds: 300

# Step 4: AI-powered bid/no-bid scoring
- name: "Score_RFP"
  type: "AI_SCORING"
```

```yaml
type: "AI_SCORING"
scoring_config:
  rules:
    - name: "Project Size Score"
      field: "gfa_sqm"
      rule_type: "range"
      parameters:
        ranges:
          - {min: 0, max: 1000, points: 5}
          - {min: 1001, max: 5000, points: 15}
          - {min: 5001, max: 20000, points: 25}
          - {min: 20001, max: 999999, points: 30}
      points: 0
    - name: "Location Preference"
      field: "location"
      rule_type: "lookup"
      parameters:
        lookup:
          "Dubai, UAE": 30
          "Abu Dhabi, UAE": 30
          "Riyadh, Saudi Arabia": 25
          "Doha, Qatar": 20
          "Other": 10
      points: 0
    - name: "Client Relationship"
      field: "client_name"
      rule_type: "custom"
      parameters:
        function: "b_bidding.scoring.check_client_history"
      points: 0
    - name: "Disciplines Match"
      field: "disciplines_required"
      rule_type: "custom"
      parameters:
        function: "b_bidding.scoring.check_discipline_capa
      points: 0
  ai_enhancement:
    enabled: true
    model: "gpt-4o"
    prompt: |
      Analyze this RFP for strategic value and potential r
      - Project complexity and innovation
      - Alignment with company expertise
      - Competitive landscape
      - Fee potential vs effort required

      Provide additional points (0-20) and detailed reasor
  recommendation_thresholds:
    A: 80  # BID - High priority
    B: 60  # BID - Conditional
    C: 40  # NO-BID - Marginal
    D: 0   # NO-BID - Clear decline
```

```
    dependencies: ["Extract_RFP_Data"]
    automate_next: true

  # Step 5: Human review and approval
  - name: "Human_Review"
    type: "STANDARD"
    function: "b_bidding.steps.pause_for_review"
    input_schema:
      type: "object"
      properties:
        approved:
          type: "boolean"
          description: "True if decision approved, False if ov
        override_reason:
          type: "string"
          description: "Reason for overriding AI recommendatic
      required: ["approved"]
    dependencies: ["Score_RFP"]
    # This step raises WorkflowPauseDirective, user resumes vi

  # Step 6: Decision routing
  - name: "Route_Decision"
    type: "DECISION"
    function: "b_bidding.steps.route_bid_decision"
    routes:
      - condition: "state.approved == true and state.recommenc
        target: "Accept_Bid"
      - condition: "state.approved == false or state.recommenc
        target: "Decline_Bid"
    dependencies: ["Human_Review"]

  # Step 7a: Accept bid - trigger sub-workflows
  - name: "Accept_Bid"
    type: "STANDARD"
    function: "b_bidding.steps.accept_bid"
    automate_next: true

  # Step 7b: Decline bid - generate email
  - name: "Decline_Bid"
    type: "STANDARD"
    function: "b_bidding.steps.decline_bid"
    automate_next: true

  # Step 8: Trigger consultant invitation sub-workflow (if acc
  - name: "Invite_Consultants"
    type: "FIRE_AND_FORGET"
    target_workflow_type: "Consultant_Invitation"
    initial_data_template:
      rfp_id: "{{state.rfp_id}}"
      project_name: "{{state.project_name}}"
      disciplines_required: "{{state.disciplines_required}}"
      submission_deadline: "{{state.submission_deadline}}"
```

```
    dependencies: ["Accept_Bid"]
```

State Model:

```python
# b_bidding/models.py
from pydantic import BaseModel
from typing import List, Optional, Dict, Any
from datetime import datetime

class RFPIntakeState(BaseModel):
    # Input
    uploaded_file: Optional[str] = None
    rfp_package_s3_key: Optional[str] = None

    # File processing
    processed_files: List[Dict[str, Any]] = []

    # Extracted data
    project_name: Optional[str] = None
    project_description: Optional[str] = None
    gfa_sqm: Optional[float] = None
    bua_sqm: Optional[float] = None
    num_buildings: Optional[int] = None
    plot_size_sqm: Optional[float] = None
    typologies: List[str] = []
    client_name: Optional[str] = None
    location: Optional[str] = None
    submission_deadline: Optional[datetime] = None
    disciplines_required: List[str] = []

    # Scoring
    score: Optional[int] = None
    recommendation: Optional[str] = None  # A, B, C, D
    scoring_reasons: List[str] = []
    ai_enhancement_points: Optional[int] = None
    ai_enhancement_reasoning: Optional[str] = None

    # Human review
    approved: Optional[bool] = None
    override_reason: Optional[str] = None
    reviewer_id: Optional[str] = None
    review_timestamp: Optional[datetime] = None

    # Outcome
    status: str = "INTAKE"  # INTAKE, ACCEPTED, DECLINED
    rfp_id: Optional[str] = None
```

3.2 Workflow: Consultant_Invitation

Purpose: Invite sub-consultants, track submissions, send reminders.

Workflow YAML (`config/workflows/consultant_invitation.yaml`):

```yaml
workflow_type: "Consultant_Invitation"
workflow_version: "1.0.0"
initial_state_model: "b_bidding.models.ConsultantInvitationStc
description: "Invite sub-consultants and track bid submissions

steps:
  # Step 1: Select consultants based on disciplines
  - name: "Select_Consultants"
    type: "STANDARD"
    function: "b_bidding.steps.select_consultants"
    input_schema:
      type: "object"
      properties:
        disciplines_required:
          type: "array"
          items:
            type: "string"
        num_per_discipline:
          type: "integer"
          default: 3
      required: ["disciplines_required"]
    automate_next: true

  # Step 2: Generate RFP packages per consultant (parallel)
  - name: "Generate_Packages"
    type: "PARALLEL"
    tasks:
      - name: "consultant_{{index}}"
        function_path: "b_bidding.steps.generate_consultant_pc
        input_template:
          consultant: "{{state.selected_consultants[index]}}"
          rfp_data: "{{state.rfp_data}}"
    merge_strategy: "SHALLOW"
    allow_partial_success: true
    dependencies: ["Select_Consultants"]
    automate_next: true

  # Step 3: Send invitations (parallel email notifier)
  - name: "Send_Invitations"
    type: "PARALLEL"
    tasks:
      - name: "email_{{index}}"
        function_path: "b_bidding.steps.send_invitation_email"
        input_template:
          consultant: "{{state.selected_consultants[index]}}"
          package_s3_key: "{{state.consultant_packages[index].
    merge_strategy: "SHALLOW"
    dependencies: ["Generate_Packages"]
    automate_next: true

  # Step 4: Schedule reminder (CRON)
  - name: "Schedule Reminder"
```

```yaml
    type: "CRON_SCHEDULER"
    target_workflow_type: "Consultant_Reminder"
    schedule: "0 9 * * *"  # Daily at 9 AM
    initial_data_template:
      rfp_id: "{{state.rfp_id}}"
      consultants: "{{state.selected_consultants}}"
      deadline: "{{state.submission_deadline}}"
    schedule_name: "reminder_{{state.rfp_id}}"
    dependencies: ["Send_Invitations"]
    automate_next: true

  # Step 5: Wait for submissions (pause until external API cal
  - name: "Track_Submissions"
    type: "STANDARD"
    function: "b_bidding.steps.pause_for_submissions"
    dependencies: ["Schedule_Reminder"]
    # This step raises WorkflowPauseDirective
    # External API calls resume workflow when consultants subm

  # Step 6: Analyze submissions (when all received or deadline
  - name: "Analyze_Submissions"
    type: "PARALLEL"
    tasks:
      - name: "analyze_{{index}}"
        function_path: "b_bidding.steps.analyze_submission"
        input_template:
          submission: "{{state.submissions[index]}}"
    merge_strategy: "DEEP"
    dependencies: ["Track_Submissions"]
    automate_next: true

  # Step 7: Generate comparison report
  - name: "Generate_Comparison"
    type: "STANDARD"
    function: "b_bidding.steps.generate_comparison_report"
    dependencies: ["Analyze_Submissions"]
```

3.3 Workflow: Partner_Prequalification (PQQ)

Purpose: Prequalify new sub-consultants via AI-assisted questionnaire review.

Workflow YAML (`config/workflows/partner_pqq.yaml`):

```yaml
workflow_type: "Partner_PQQ"
workflow_version: "1.0.0"
initial_state_model: "b_bidding.models.PartnerPQQState"
description: "Partner consultant prequalification workflow"

steps:
  - name: "Submit_Questionnaire"
    type: "STANDARD"
    function: "b_bidding.steps.receive_pqq_questionnaire"
    automate_next: true

  - name: "AI_Assessment"
    type: "AI_SCORING"
    scoring_config:
      rules:
        - name: "Experience Score"
          field: "years_experience"
          rule_type: "range"
          parameters:
            ranges:
              - {min: 0, max: 5, points: 10}
              - {min: 6, max: 10, points: 20}
              - {min: 11, max: 999, points: 30}
        - name: "Project Portfolio"
          field: "num_completed_projects"
          rule_type: "range"
          parameters:
            ranges:
              - {min: 0, max: 10, points: 10}
              - {min: 11, max: 50, points: 25}
              - {min: 51, max: 999, points: 40}
        - name: "Certifications"
          field: "certifications"
          rule_type: "custom"
          parameters:
            function: "b_bidding.scoring.check_certifications"
      recommendation_thresholds:
        APPROVED: 70
        CONDITIONAL: 50
        REJECTED: 0
    dependencies: ["Submit_Questionnaire"]
    automate_next: true

  - name: "Human_Review_PQQ"
    type: "STANDARD"
    function: "b_bidding.steps.pause_for_pqq_review"
    dependencies: ["AI_Assessment"]

  - name: "Route_PQQ_Decision"
    type: "DECISION"
    function: "b_bidding.steps.route_pqq_decision"
    routes:
```

```yaml
      - condition: "state.pqq_approved == true"
        target: "Approve_Partner"
      - condition: "state.pqq_approved == false"
        target: "Reject_Partner"
    dependencies: ["Human_Review_PQQ"]

  - name: "Approve_Partner"
    type: "STANDARD"
    function: "b_bidding.steps.approve_partner"
    automate_next: true

  - name: "Reject_Partner"
    type: "EMAIL_NOTIFIER"
    email_config:
      template:
        subject: "PQQ Status: Not Approved"
        body: |
          Dear {{state.partner_name}},

          Thank you for your interest. Unfortunately, your pre
          did not meet our criteria at this time.

          Feedback: {{state.rejection_feedback}}
        recipients: ["{{state.partner_email}}"]
      smtp_config:
        provider: "sendgrid"
        api_key: "{{env.SENDGRID_API_KEY}}"
```

3.4 Workflow: Contract_Review

Purpose: AI-powered contract analysis with White Book comparison.

Workflow YAML (`config/workflows/contract_review.yaml`):

```yaml
workflow_type: "Contract_Review"
workflow_version: "1.0.0"
initial_state_model: "b_bidding.models.ContractReviewState"
description: "AI contract review with risk flagging"

steps:
  - name: "Upload_Contract"
    type: "STANDARD"
    function: "b_bidding.steps.upload_contract"
    automate_next: true

  - name: "AI_Contract_Analysis"
    type: "AI_CONTRACT_REVIEW"
    contract_config:
      document_keys: ["contract_pdf_s3_key"]
      clauses_to_flag:
        - name: "Performance Bond"
          keywords: ["performance bond", "bank guarantee", "fi
          risk_level: "HIGH"
          description: "Financial security required from contr
          whitebook_reference: "Section 4.2"
        - name: "Delay Penalties / LDs"
          keywords: ["liquidated damages", "delay penalty", "l
          risk_level: "CRITICAL"
          description: "Financial penalties for project delays
          whitebook_reference: "Section 5.3"
        - name: "Tender Bond"
          keywords: ["tender bond", "bid bond"]
          risk_level: "MEDIUM"
          description: "Security required with bid submission"
          whitebook_reference: "Section 3.1"
        - name: "Professional Indemnity Insurance"
          keywords: ["professional indemnity", "PI insurance",
          risk_level: "HIGH"
          description: "Insurance coverage requirements"
          whitebook_reference: "Section 6.4"
      whitebook_path: "whitebook/fidic_white_book_2023.pdf"
      comparison_enabled: true
      llm_provider: "anthropic"
      model: "claude-3-5-sonnet-20241022"
    dependencies: ["Upload_Contract"]
    automate_next: true
    timeout_seconds: 300

  - name: "Generate_Risk_Report"
    type: "TEMPLATE_GENERATOR"
    template_config:
      template_s3_key: "templates/contract_risk_report_v1.docx
      output_filename: "Contract_Risk_Report_{{state.project_r
      data_mappings:
        "{{PROJECT_NAME}}": "state.project_name"
        "{{CLIENT_NAME}}": "state.client_name"
```

```yaml
        "{{FLAGGED_CLAUSES}}": "state.flagged_clauses"  # List
        "{{RISK_SUMMARY}}": "state.risk_summary"
      format: "docx"
      output_key: "risk_report_s3_key"
    dependencies: ["AI_Contract_Analysis"]
    automate_next: true

  - name: "Human_Review_Contract"
    type: "STANDARD"
    function: "b_bidding.steps.pause_for_contract_review"
    dependencies: ["Generate_Risk_Report"]

  - name: "Route_Contract_Decision"
    type: "DECISION"
    function: "b_bidding.steps.route_contract_decision"
    routes:
      - condition: "state.contract_acceptable == true"
        target: "Accept_Contract"
      - condition: "state.contract_acceptable == false"
        target: "Negotiate_Contract"
    dependencies: ["Human_Review_Contract"]

  - name: "Accept_Contract"
    type: "STANDARD"
    function: "b_bidding.steps.accept_contract"

  - name: "Negotiate_Contract"
    type: "STANDARD"
    function: "b_bidding.steps.initiate_negotiation"
```

3.5 Workflow Dependency Graph

```
RFP_Intake
    |
    ├─► [ACCEPTED] ─► Consultant_Invitation
    |                      |
    |                      ├─► Consultant_Reminder (CRON, recu
    |                      └─► (Multiple consultants submit bi
    |
    ├─► [DECLINED] ─► (End)
    |
    └─► Contract_Review
            |
            ├─► [ACCEPTABLE] ─► Bid_Submission_Workflow
            └─► [NEGOTIATE] ─► Contract_Negotiation_Workflow
```

---

4. FastAPI API Design

4.1 API Structure

```
/api/v1/
├── /auth/
|   ├── POST /login                 # JWT authentication
|   ├── POST /refresh               # Refresh token
|   └── POST /logout                # Invalidate token
|
├── /rfps/
|   ├── POST /upload                # Upload RFP package
|   ├── GET /                       # List all RFPs
|   ├── GET /{rfp_id}               # Get RFP details
|   ├── PUT /{rfp_id}               # Update RFP metadata
|   └── DELETE /{rfp_id}            # Delete RFP
|
├── /workflows/
|   ├── POST /start                 # Start a workflow
|   ├── GET /                       # List workflows (filterc
|   ├── GET /{workflow_id}          # Get workflow details
|   ├── POST /{workflow_id}/resume  # Resume paused workflow
|   ├── POST /{workflow_id}/retry   # Retry failed workflow
|   ├── POST /{workflow_id}/cancel  # Cancel workflow
|   ├── GET /{workflow_id}/logs     # Get execution logs
|   └── GET /{workflow_id}/state    # Get current state
|
├── /partners/
|   ├── POST /                      # Create partner
|   ├── GET /                       # List partners
|   ├── GET /{partner_id}           # Get partner details
|   ├── PUT /{partner_id}           # Update partner
|   ├── POST /{partner_id}/pqq      # Submit PQQ
|   ├── GET /{partner_id}/submissions # Get partner's bid hist
|   └── POST /{partner_id}/portal-access # Generate portal acc
|
├── /bids/
|   ├── POST /                      # Submit bid (partner por
|   ├── GET /                       # List bids
|   ├── GET /{bid_id}               # Get bid details
|   ├── PUT /{bid_id}               # Update bid
|   ├── POST /{bid_id}/review       # Submit bid review
|   └── GET /{bid_id}/comparison    # Get comparison with otl
|
├── /contracts/
|   ├── POST /upload                # Upload contract
|   ├── GET /                       # List contracts
|   ├── GET /{contract_id}          # Get contract details
|   ├── GET /{contract_id}/analysis # Get AI analysis
|   └── POST /{contract_id}/review  # Submit contract decisic
|
├── /analytics/
|   ├── GET /dashboard              # Dashboard metrics
|   ├── GET /win-rate               # Win/loss analysis
|   ├── GET /partner-performance    # Partner metrics
|   ├── GET /fee-analysis           # Fee per sam. % of const
```

```
|   └── GET /export                    # Export data (CSV/Excel)
|
└── /admin/
    ├── GET /settings                  # System settings
    ├── PUT /settings                  # Update settings
    ├── GET /audit-logs                # Audit trail
    └── POST /zombie-scan              # Trigger zombie workflow
```

4.2 Example API Endpoints

POST /api/v1/rfps/upload

Request:

```http
POST /api/v1/rfps/upload HTTP/1.1
Content-Type: multipart/form-data
Authorization: Bearer <jwt_token>

file: <binary data>
metadata: {
  "source": "email",
  "received_date": "2026-02-11T10:00:00Z",
  "notes": "High priority RFP from Dubai client"
}
```

Response:

```json
{
  "rfp_id": "rfp_20260211_001",
  "workflow_id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "PROCESSING",
  "s3_key": "rfps/2026-02-11/rfp_20260211_001.zip",
  "created_at": "2026-02-11T10:05:23Z"
}
```

Backend Logic (FastAPI route handler):

```python
# src/b_bidding_api/routes/rfps.py
from fastapi import APIRouter, UploadFile, File, Depends, HTTP
from rufus.builder import WorkflowBuilder
from b_bidding_api.dependencies import get_workflow_builder
from b_bidding_api.storage import upload_to_s3
import uuid

router = APIRouter()

@router.post("/upload")
async def upload_rfp(
    file: UploadFile = File(...),
    workflow_builder: WorkflowBuilder = Depends(get_workflow_b
):
    # Generate unique ID
    rfp_id = f"rfp_{datetime.now().strftime('%Y%m%d')}_{uuid.u

    # Upload to S3
    s3_key = await upload_to_s3(
        file=file.file,
        bucket="b-bidding-rfps",
        key=f"rfps/{datetime.now().strftime('%Y-%m-%d')}/{rfp_
    )

    # Start Rufus workflow
    workflow = await workflow_builder.create_workflow(
        workflow_type="RFP_Intake",
        initial_data={
            "rfp_id": rfp_id,
            "rfp_package_s3_key": s3_key,
            "uploaded_at": datetime.now().isoformat(),
            "uploaded_by": "user_id_from_jwt"  # Extract from
        }
    )

    # Trigger first step (Store_RFP_Package)
    await workflow.next_step()

    return {
        "rfp_id": rfp_id,
        "workflow_id": str(workflow.id),
        "status": workflow.status.value,
        "s3_key": s3_key,
        "created_at": workflow.created_at.isoformat()
    }
```

POST /api/v1/workflows/{workflow_id}/resume

Request:

```
POST /api/v1/workflows/550e8400-e29b-41d4-a716-446655440000/re
Content-Type: application/json
Authorization: Bearer <jwt_token>

{
  "approved": true,
  "override_reason": null,
  "reviewer_id": "bronwyn_user_id"
}
```

Response:

```
{
  "workflow_id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "ACTIVE",
  "current_step": "Route_Decision",
  "state": {
    "project_name": "Dubai Marina Tower",
    "score": 85,
    "recommendation": "A",
    "approved": true
  }
}
```

Backend Logic:

```python
@router.post("/{workflow_id}/resume")
async def resume_workflow(
    workflow_id: UUID,
    resume_data: dict,
    workflow_builder: WorkflowBuilder = Depends(get_workflow_b
):
    # Load workflow from persistence
    workflow = await workflow_builder.load_workflow(workflow_i

    if workflow.status != WorkflowStatus.WAITING_HUMAN:
        raise HTTPException(status_code=400, detail="Workflow

    # Resume with user input
    result = await workflow.next_step(user_input=resume_data)

    return {
        "workflow_id": str(workflow.id),
        "status": workflow.status.value,
        "current_step": workflow.workflow_steps[workflow.curre
        "state": workflow.state.model_dump()
    }
```

GET /api/v1/analytics/dashboard

Response:


```json
{
  "total_rfps": 150,
  "active_bids": 23,
  "pending_reviews": 5,
  "win_rate": 0.42,
  "avg_response_time_hours": 18.5,
  "top_clients": [
    {"name": "Emaar Properties", "rfps_count": 15, "win_rate":
    {"name": "Dubai Holding", "rfps_count": 12, "win_rate": 0.
  ],
  "pipeline_value_usd": 25000000,
  "recent_activity": [
    {
      "rfp_id": "rfp_20260211_001",
      "project_name": "Dubai Marina Tower",
      "status": "ACCEPTED",
      "timestamp": "2026-02-11T14:23:00Z"
    }
  ]
}
```

4.3 Authentication & Authorization

JWT Strategy:

```python
# src/b_bidding_api/auth.py
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCrec
from jose import JWTError, jwt
from datetime import datetime, timedelta

SECRET_KEY = "your-secret-key"  # Use environment variable
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60

security = HTTPBearer()

def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKE
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=
    return encoded_jwt

def verify_token(credentials: HTTPAuthorizationCredentials = D
    try:
        payload = jwt.decode(credentials.credentials, SECRET_K
        user_id = payload.get("sub")
        role = payload.get("role")  # "admin", "internal_user'
        if user_id is None:
            raise HTTPException(status_code=401, detail="Inval
        return {"user_id": user_id, "role": role}
    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid t

# Role-based access control
def require_role(required_role: str):
    def role_checker(user: dict = Depends(verify_token)):
        if user["role"] != required_role:
            raise HTTPException(status_code=403, detail="Insuf
        return user
    return role_checker
```

Usage in routes:

```python
@router.post("/upload")
async def upload_rfp(
    file: UploadFile = File(...),
    user: dict = Depends(require_role("internal_user"))  # Onl
):
    ...
```

4.4 WebSocket Support (Real-Time Updates)

For live workflow status updates:

```python
# src/b_bidding_api/websocket.py
from fastapi import WebSocket, WebSocketDisconnect
from typing import Dict
import json

class ConnectionManager:
    def __init__(self):
        self.active_connections: Dict[str, WebSocket] = {}

    async def connect(self, workflow_id: str, websocket: WebSc
        await websocket.accept()
        self.active_connections[workflow_id] = websocket

    def disconnect(self, workflow_id: str):
        if workflow_id in self.active_connections:
            del self.active_connections[workflow_id]

    async def send_update(self, workflow_id: str, message: dic
        if workflow_id in self.active_connections:
            await self.active_connections[workflow_id].send_te

manager = ConnectionManager()

@app.websocket("/ws/workflows/{workflow_id}")
async def workflow_updates(websocket: WebSocket, workflow_id:
    await manager.connect(workflow_id, websocket)
    try:
        while True:
            # Keep connection alive
            await websocket.receive_text()
    except WebSocketDisconnect:
        manager.disconnect(workflow_id)

# In WorkflowObserver implementation
class WebSocketObserver:
    async def on_workflow_status_changed(self, workflow_id, ol
        await manager.send_update(str(workflow_id), {
            "event": "status_changed",
            "old_status": old_status.value,
            "new_status": new_status.value,
            "timestamp": datetime.now().isoformat()
        })
```

---

5. Frontend Architecture

5.1 Technology Stack

Core:

- **React 18** with TypeScript
- **Vite** for build tooling (fast HMR)
- **React Router v6** for navigation
- **TanStack Query (React Query)** for data fetching
- **Zustand** for global state (lightweight alternative to Redux)

UI Components:

- **Tailwind CSS** for styling
- **shadcn/ui** (Radix UI primitives) for components
- **Recharts** for analytics dashboards
- **React Dropzone** for file uploads
- **React Table (TanStack Table)** for data grids

Forms & Validation:

- **React Hook Form** for form management
- **Zod** for schema validation

Utilities:

- **Axios** for HTTP requests
- **date-fns** for date formatting
- **react-toastify** for notifications

5.2 Application Structure

```
frontend/
├── public/
├── src/
|   ├── main.tsx                  # Entry point
|   ├── App.tsx                   # Root component
|   ├── routes.tsx                # Route definitions
|   |
|   ├── features/                 # Feature-based organizatior
|   |   ├── auth/
|   |   |   ├── LoginPage.tsx
|   |   |   ├── AuthContext.tsx
|   |   |   └── hooks/useAuth.ts
|   |   |
|   |   ├── rfps/
|   |   |   ├── RFPListPage.tsx
|   |   |   ├── RFPDetailPage.tsx
|   |   |   ├── RFPUploadPage.tsx
|   |   |   ├── components/
|   |   |   |   ├── RFPCard.tsx
|   |   |   |   ├── RFPTable.tsx
|   |   |   |   └── ExtractedFieldsDisplay.tsx
|   |   |   └── hooks/
|   |   |       ├── useRFPs.ts
|   |   |       └── useRFPUpload.ts
|   |   |
|   |   ├── workflows/
|   |   |   ├── WorkflowListPage.tsx
|   |   |   ├── WorkflowDetailPage.tsx
|   |   |   ├── components/
|   |   |   |   ├── WorkflowStatusBadge.tsx
|   |   |   |   ├── WorkflowTimeline.tsx
|   |   |   |   └── ApprovalForm.tsx
|   |   |   └── hooks/
|   |   |       ├── useWorkflows.ts
|   |   |       └── useWorkflowResume.ts
|   |   |
|   |   ├── partners/
|   |   |   ├── PartnerListPage.tsx
|   |   |   ├── PartnerDetailPage.tsx
|   |   |   ├── PartnerPortalPage.tsx  # Partner-facing
|   |   |   ├── PQQFormPage.tsx        # Partner-facing
|   |   |   └── components/
|   |   |       ├── PartnerCard.tsx
|   |   |       ├── PQQForm.tsx
|   |   |       └── BidSubmissionForm.tsx
|   |   |
|   |   ├── analytics/
|   |   |   ├── DashboardPage.tsx
|   |   |   ├── WinRateAnalysisPage.tsx
|   |   |   ├── PartnerPerformancePage.tsx
|   |   |   └── components/
|   |   |       ├── MetricCard.tsx
```

```
|   |   |       ├── WinLossChart.tsx
|   |   |       └── FeeAnalysisChart.tsx
|   |   |
|   |   └── contracts/
|   |       ├── ContractListPage.tsx
|   |       ├── ContractReviewPage.tsx
|   |       └── components/
|   |           ├── ClauseFlagging.tsx
|   |           └── RiskSummary.tsx
|   |
|   ├── components/               # Shared components
|   |   ├── ui/                   # shadcn/ui components
|   |   |   ├── Button.tsx
|   |   |   ├── Card.tsx
|   |   |   ├── Dialog.tsx
|   |   |   └── ...
|   |   ├── Layout/
|   |   |   ├── DashboardLayout.tsx
|   |   |   ├── PortalLayout.tsx
|   |   |   ├── Sidebar.tsx
|   |   |   └── Header.tsx
|   |   ├── FileUpload.tsx
|   |   ├── DataTable.tsx
|   |   └── LoadingSpinner.tsx
|   |
|   ├── lib/                      # Utilities
|   |   ├── api.ts                # Axios instance + intercept
|   |   ├── queryClient.ts        # React Query config
|   |   ├── websocket.ts          # WebSocket client
|   |   └── utils.ts              # Helper functions
|   |
|   ├── hooks/                    # Shared hooks
|   |   ├── useAuth.ts
|   |   ├── useWebSocket.ts
|   |   └── useFileUpload.ts
|   |
|   ├── stores/                   # Zustand stores
|   |   ├── authStore.ts
|   |   └── notificationStore.ts
|   |
|   ├── types/                    # TypeScript types
|   |   ├── rfp.types.ts
|   |   ├── workflow.types.ts
|   |   ├── partner.types.ts
|   |   └── api.types.ts
|   |
|   └── styles/
|       └── globals.css           # Tailwind imports
|
├── package.json
├── tsconfig.json
└── vite.config.ts
```

```
├── vite.config.ts
└── tailwind.config.js
```

5.3 Key UI Components

5.3.1 RFP Upload Page

```
// src/features/rfps/RFPUploadPage.tsx
import { useState } from 'react';
import { useRFPUpload } from './hooks/useRFPUpload';
import { FileUpload } from '@/components/FileUpload';
import { Button } from '@/components/ui/Button';
import { useNavigate } from 'react-router-dom';

export function RFPUploadPage() {
  const [file, setFile] = useState<File | null>(null);
  const navigate = useNavigate();
  const uploadMutation = useRFPUpload();

  const handleUpload = async () => {
    if (!file) return;

    const result = await uploadMutation.mutateAsync(file);

    // Navigate to workflow detail page
    navigate(`/workflows/${result.workflow_id}`);
  };

  return (
    <div className="container mx-auto p-6">
      <h1 className="text-3xl font-bold mb-6">Upload RFP</h1>

      <FileUpload
        onFileSelect={setFile}
        accept=".zip,.pdf,.docx"
        maxSizeMB={10000}
      />

      {file && (
        <div className="mt-4">
          <p className="text-sm text-gray-600">
            Selected: {file.name} ({(file.size / 1024 / 1024).
          </p>
          <Button
            onClick={handleUpload}
            disabled={uploadMutation.isLoading}
            className="mt-2"
          >
            {uploadMutation.isLoading ? 'Uploading...' : 'Uplc
          </Button>
        </div>
      )}
    </div>
  );
}

// src/features/rfps/hooks/useRFPUpload.ts
import { useMutation } from '@tanstack/react-query';
import { api } from '@/lib/api';
```

```
export function useRFPUpload() {
  return useMutation({
    mutationFn: async (file: File) => {
      const formData = new FormData();
      formData.append('file', file);

      const response = await api.post('/api/v1/rfps/upload', f
        headers: { 'Content-Type': 'multipart/form-data' }
      });

      return response.data;
    }
  });
}
```

5.3.2 Workflow Detail Page with Live Updates

```
// src/features/workflows/WorkflowDetailPage.tsx
import { useParams } from 'react-router-dom';
import { useWorkflow } from './hooks/useWorkflow';
import { useWebSocket } from '@/hooks/useWebSocket';
import { WorkflowTimeline } from './components/WorkflowTimelir
import { ApprovalForm } from './components/ApprovalForm';
import { Card } from '@/components/ui/Card';

export function WorkflowDetailPage() {
  const { workflowId } = useParams();
  const { data: workflow, refetch } = useWorkflow(workflowId!)

  // Real-time updates via WebSocket
  useWebSocket(`/ws/workflows/${workflowId}`, {
    onMessage: (event) => {
      const data = JSON.parse(event.data);
      if (data.event === 'status_changed') {
        refetch();  // Refetch workflow data
      }
    }
  });

  if (!workflow) return <div>Loading...</div>;

  return (
    <div className="container mx-auto p-6">
      <div className="flex justify-between items-center mb-6">
        <h1 className="text-3xl font-bold">{workflow.state.prc
        <WorkflowStatusBadge status={workflow.status} />
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
        {/* Left column: Workflow state */}
        <div className="lg:col-span-2">
          <Card className="p-6">
            <h2 className="text-xl font-semibold mb-4">Extract
            <dl className="grid grid-cols-2 gap-4">
              <div>
                <dt className="text-sm text-gray-600">Project
                <dd className="font-medium">{workflow.state.pr
              </div>
              <div>
                <dt className="text-sm text-gray-600">GFA (sqm
                <dd className="font-medium">{workflow.state.gf
              </div>
              <div>
                <dt className="text-sm text-gray-600">Locatior
                <dd className="font-medium">{workflow.state.lc
              </div>
              <div>
                <dt className="text-sm text-gray-600">Client</
                <dd class="font-medium">{workflow.state.cl
```

```
          </div>
          <div>
            <dt className="text-sm text-gray-600">Score</d
            <dd className="font-medium text-green-600">
              {workflow.state.score} / 100
            </dd>
          </div>
          <div>
            <dt className="text-sm text-gray-600">Recommer
            <dd className="font-medium">
              Grade {workflow.state.recommendation}
            </dd>
          </div>
        </dl>

        {workflow.state.scoring_reasons && (
          <div className="mt-4">
            <h3 className="font-semibold mb-2">Scoring Rec
            <ul className="list-disc pl-5 space-y-1">
              {workflow.state.scoring_reasons.map((reason,
                <li key={i} className="text-sm">{reason}</
              ))}
            </ul>
          </div>
        )}
      </Card>

      {/* Timeline */}
      <Card className="p-6 mt-6">
        <h2 className="text-xl font-semibold mb-4">Workflc
        <WorkflowTimeline
          steps={workflow.steps}
          currentStep={workflow.current_step}
        />
      </Card>
    </div>

    {/* Right column: Actions */}
    <div>
      {workflow.status === 'WAITING_HUMAN' && (
        <Card className="p-6">
          <h2 className="text-xl font-semibold mb-4">Revie
          <ApprovalForm workflowId={workflowId!} />
        </Card>
      )}

      {workflow.status === 'FAILED' && (
        <Card className="p-6 border-red-500">
          <h2 className="text-xl font-semibold mb-4 text-r
            Workflow Failed
          </h2>
          <p className="text-sm text-gray-600 mb-4">
```

```
                <p className="text-sm text-gray-600 mb-4">
                  {workflow.error_message}
                </p>
                <Button variant="outline" onClick={() => {/* Ret
                  Retry Workflow
                </Button>
              </Card>
            )}
          </div>
        </div>
      </div>
    );
  }
```

5.3.3 Analytics Dashboard

```
// src/features/analytics/DashboardPage.tsx
import { useAnalyticsDashboard } from './hooks/useAnalyticsDas
import { MetricCard } from './components/MetricCard';
import { WinLossChart } from './components/WinLossChart';
import { Card } from '@/components/ui/Card';

export function DashboardPage() {
  const { data: metrics } = useAnalyticsDashboard();

  if (!metrics) return <div>Loading...</div>;

  return (
    <div className="container mx-auto p-6">
      <h1 className="text-3xl font-bold mb-6">Analytics Dashbc

      {/* KPI Cards */}
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-
        <MetricCard
          title="Total RFPs"
          value={metrics.total_rfps}
          trend="+12%"
          icon="📄"
        />
        <MetricCard
          title="Active Bids"
          value={metrics.active_bids}
          trend="+5"
          icon="📊"
        />
        <MetricCard
          title="Win Rate"
          value={`${(metrics.win_rate * 100).toFixed(1)}%`}
          trend="+2.3%"
          icon="🏆"
        />
        <MetricCard
          title="Pipeline Value"
          value={`$${(metrics.pipeline_value_usd / 1e6).toFixe
          trend="+$1.2M"
          icon="💰"
        />
      </div>

      {/* Charts */}
      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
        <Card className="p-6">
          <h2 className="text-xl font-semibold mb-4">Win/Loss
          <WinLossChart data={metrics.win_loss_history} />
        </Card>

        <Card className="p-6">
          <h2 className="text-xl font-semibold mb-4">Top Clier
```

```
            <div className="space-y-3">
              {metrics.top_clients.map((client, i) => (
                <div key={i} className="flex justify-between ite
                  <span className="font-medium">{client.name}</s
                  <div className="text-right">
                    <div className="text-sm text-gray-600">
                      {client.rfps_count} RFPs
                    </div>
                    <div className="text-sm font-semibold text-g
                      {(client.win_rate * 100).toFixed(0)}% win
                    </div>
                  </div>
                </div>
              ))}
            </div>
          </Card>
        </div>
      </div>
    );
  }
```

5.3.4 Partner Portal (External Access)

```
// src/features/partners/PartnerPortalPage.tsx
import { useAuth } from '@/features/auth/hooks/useAuth';
import { usePartnerBids } from './hooks/usePartnerBids';
import { Card } from '@/components/ui/Card';
import { Button } from '@/components/ui/Button';

export function PartnerPortalPage() {
  const { user } = useAuth();  // Partner user
  const { data: bids } = usePartnerBids(user.partner_id);

  return (
    <div className="container mx-auto p-6">
      <h1 className="text-3xl font-bold mb-6">
        Welcome, {user.company_name}
      </h1>

      {/* Active RFP Invitations */}
      <Card className="p-6 mb-6">
        <h2 className="text-xl font-semibold mb-4">Active Invi
        <div className="space-y-4">
          {bids?.active.map((bid) => (
            <div
              key={bid.id}
              className="border rounded-lg p-4 hover:bg-gray-5
            >
              <div className="flex justify-between items-start
                <div>
                  <h3 className="font-semibold">{bid.project_r
                  <p className="text-sm text-gray-600">{bid.cl
                  <p className="text-sm text-gray-600 mt-1">
                    Deadline: {new Date(bid.deadline).toLocale
                  </p>
                </div>
                <Button
                  onClick={() => {/* Navigate to bid form */}}
                  variant="primary"
                >
                  Submit Bid
                </Button>
              </div>
            </div>
          ))}
        </div>
      </Card>

      {/* Past Submissions */}
      <Card className="p-6">
        <h2 className="text-xl font-semibold mb-4">Past Submis
        <table className="w-full">
          <thead>
            <tr className="border-b">
              <th className="text-left py-2">Project</th>
```

```
                <th className="text-left py-2">Client</th>
                <th className="text-left py-2">Submitted</th>
                <th className="text-left py-2">Status</th>
              </tr>
            </thead>
            <tbody>
              {bids?.past.map((bid) => (
                <tr key={bid.id} className="border-b">
                  <td className="py-2">{bid.project_name}</td>
                  <td className="py-2">{bid.client_name}</td>
                  <td className="py-2">
                    {new Date(bid.submitted_at).toLocaleDateStri
                  </td>
                  <td className="py-2">
                    <span className={`px-2 py-1 rounded text-sm
                      bid.status === 'WON' ? 'bg-green-100 text-
                      bid.status === 'LOST' ? 'bg-red-100 text-r
                      'bg-yellow-100 text-yellow-800'
                    }`}>
                      {bid.status}
                    </span>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        </Card>
      </div>
    );
  }
```

5.4 State Management

Zustand Store Example:

```
// src/stores/authStore.ts
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

interface User {
  id: string;
  email: string;
  role: 'admin' | 'internal_user' | 'partner';
  company_name?: string;
  partner_id?: string;
}

interface AuthState {
  user: User | null;
  token: string | null;
  login: (email: string, password: string) => Promise<void>;
  logout: () => void;
  setUser: (user: User, token: string) => void;
}

export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      user: null,
      token: null,
      login: async (email, password) => {
        // Call API
        const response = await api.post('/api/v1/auth/login',
        set({ user: response.data.user, token: response.data.t
      },
      logout: () => {
        set({ user: null, token: null });
      },
      setUser: (user, token) => {
        set({ user, token });
      }
    }),
    {
      name: 'auth-storage',  // LocalStorage key
    }
  )
);
```

---

6. Database Schema

6.1 Core Tables (Beyond Rufus SDK Tables)

```sql
-- RFPs table
CREATE TABLE rfps (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    rfp_id VARCHAR(100) UNIQUE NOT NULL,  -- Human-readable Il
    workflow_id UUID REFERENCES workflow_executions(id) ON DEL

    -- Metadata
    project_name VARCHAR(500),
    project_description TEXT,
    client_name VARCHAR(200),
    location VARCHAR(200),
    submission_deadline TIMESTAMPTZ,

    -- Extracted fields
    gfa_sqm NUMERIC,
    bua_sqm NUMERIC,
    num_buildings INTEGER,
    plot_size_sqm NUMERIC,
    typologies JSONB,  -- Array of strings
    disciplines_required JSONB,  -- Array of strings

    -- Scoring
    score INTEGER,
    recommendation VARCHAR(10),  -- A, B, C, D
    scoring_reasons JSONB,  -- Array of strings

    -- Status
    status VARCHAR(50) DEFAULT 'INTAKE',  -- INTAKE, ACCEPTED,
    approved BOOLEAN,
    override_reason TEXT,
    reviewer_id UUID REFERENCES users(id),
    review_timestamp TIMESTAMPTZ,

    -- File storage
    rfp_package_s3_key VARCHAR(500),
    processed_files JSONB,  -- Array of file metadata

    -- Audit
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),
    created_by UUID REFERENCES users(id),

    -- Full-text search
    search_vector tsvector
);

CREATE INDEX idx_rfps_status ON rfps(status);
CREATE INDEX idx_rfps_workflow_id ON rfps(workflow_id);
CREATE INDEX idx_rfps_client_name ON rfps(client_name);
CREATE INDEX idx_rfps_submission_deadline ON rfps(submission_c
CREATE INDEX idx_rfps_search ON rfps USING GIN(search_vector);
```

```sql
-- Trigger for full-text search
CREATE TRIGGER rfps_search_update
BEFORE INSERT OR UPDATE ON rfps
FOR EACH ROW EXECUTE FUNCTION
tsvector_update_trigger(search_vector, 'pg_catalog.english',
    project_name, project_description, client_name, location);

-- Partners/Consultants table
CREATE TABLE partners (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    partner_id VARCHAR(100) UNIQUE NOT NULL,

    -- Company info
    company_name VARCHAR(200) NOT NULL,
    registration_number VARCHAR(100),
    country VARCHAR(100),
    city VARCHAR(100),

    -- Contact
    primary_contact_name VARCHAR(200),
    primary_contact_email VARCHAR(200) UNIQUE NOT NULL,
    primary_contact_phone VARCHAR(50),

    -- Prequalification
    pqq_status VARCHAR(50) DEFAULT 'PENDING',  -- PENDING, APP
    pqq_workflow_id UUID REFERENCES workflow_executions(id),
    pqq_score INTEGER,
    pqq_approved_at TIMESTAMPTZ,

    -- Capabilities
    disciplines JSONB,  -- Array of disciplines
    certifications JSONB,  -- Array of certifications
    years_experience INTEGER,
    num_completed_projects INTEGER,
    typical_project_size VARCHAR(50),

    -- Performance tracking
    total_bids INTEGER DEFAULT 0,
    total_wins INTEGER DEFAULT 0,
    win_rate NUMERIC,
    avg_response_time_hours NUMERIC,

    -- Portal access
    portal_access_enabled BOOLEAN DEFAULT false,
    last_login_at TIMESTAMPTZ,

    -- Audit
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_partners_pqq_status ON partners(pqq_status);
```

```
CREATE INDEX idx_partners_pqq_status ON partners(pqq_status);
CREATE INDEX idx_partners_disciplines ON partners USING GIN(di
CREATE INDEX idx_partners_email ON partners(primary_contact_em

-- Bids table
CREATE TABLE bids (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    bid_id VARCHAR(100) UNIQUE NOT NULL,

    -- References
    rfp_id UUID REFERENCES rfps(id) ON DELETE CASCADE,
    partner_id UUID REFERENCES partners(id) ON DELETE CASCADE,
    invitation_workflow_id UUID REFERENCES workflow_executions

    -- Bid details
    discipline VARCHAR(100),
    proposed_fee NUMERIC,
    currency VARCHAR(10) DEFAULT 'USD',
    fee_breakdown JSONB,  -- Detailed cost breakdown
    exclusions TEXT,
    notes TEXT,

    -- Submission
    submitted_at TIMESTAMPTZ,
    submission_method VARCHAR(50),  -- PORTAL, EMAIL

    -- Status
    status VARCHAR(50) DEFAULT 'INVITED',  -- INVITED, SUBMITT

    -- Review
    reviewed_at TIMESTAMPTZ,
    reviewed_by UUID REFERENCES users(id),
    review_notes TEXT,

    -- Attachments
    attachment_s3_keys JSONB,  -- Array of S3 keys

    -- Audit
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_bids_rfp_id ON bids(rfp_id);
CREATE INDEX idx_bids_partner_id ON bids(partner_id);
CREATE INDEX idx_bids_status ON bids(status);
CREATE INDEX idx_bids_submitted_at ON bids(submitted_at);

-- Contracts table
CREATE TABLE contracts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    contract_id VARCHAR(100) UNIQUE NOT NULL,
```

```sql
    -- References
    rfp_id UUID REFERENCES rfps(id) ON DELETE SET NULL,
    workflow_id UUID REFERENCES workflow_executions(id),

    -- Contract details
    contract_type VARCHAR(100),  -- FIDIC, NEC, Custom
    client_name VARCHAR(200),
    project_name VARCHAR(500),

    -- File storage
    contract_pdf_s3_key VARCHAR(500),

    -- AI Analysis
    flagged_clauses JSONB,  -- Array of {clause, risk_level, c
    risk_summary TEXT,
    whitebook_comparison JSONB,
    overall_risk_level VARCHAR(50),  -- LOW, MEDIUM, HIGH, CRI

    -- Review
    contract_acceptable BOOLEAN,
    review_notes TEXT,
    reviewed_at TIMESTAMPTZ,
    reviewed_by UUID REFERENCES users(id),

    -- Generated reports
    risk_report_s3_key VARCHAR(500),

    -- Audit
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_contracts_rfp_id ON contracts(rfp_id);
CREATE INDEX idx_contracts_workflow_id ON contracts(workflow_i
CREATE INDEX idx_contracts_risk_level ON contracts(overall_ris

-- Analytics metrics (materialized view or table)
CREATE TABLE analytics_metrics (
    id SERIAL PRIMARY KEY,
    metric_date DATE NOT NULL,

    -- RFP metrics
    total_rfps_received INTEGER,
    rfps_accepted INTEGER,
    rfps_declined INTEGER,

    -- Bid metrics
    total_bids_submitted INTEGER,
    bids_won INTEGER,
    bids_lost INTEGER,
    win_rate NUMERIC,
```

```sql
    -- Financial metrics
    total_bid_value_usd NUMERIC,
    won_bid_value_usd NUMERIC,
    avg_fee_per_sqm NUMERIC,

    -- Performance metrics
    avg_response_time_hours NUMERIC,
    avg_decision_time_hours NUMERIC,

    -- Partner metrics
    active_partners INTEGER,
    new_partners INTEGER,

    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_analytics_date ON analytics_metrics(metric_dc

-- Users table (for internal users + partner portal users)
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(200) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,

    -- Profile
    first_name VARCHAR(100),
    last_name VARCHAR(100),
    role VARCHAR(50) NOT NULL,  -- admin, internal_user, partr

    -- Partner link (if role = partner)
    partner_id UUID REFERENCES partners(id),

    -- Auth
    last_login_at TIMESTAMPTZ,
    password_reset_token VARCHAR(255),
    password_reset_expires_at TIMESTAMPTZ,

    -- Audit
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),
    is_active BOOLEAN DEFAULT true
);

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_role ON users(role);
```

6.2 Database Views

```sql
-- View: RFP Dashboard Summary
CREATE VIEW v_rfp_dashboard AS
SELECT
    DATE_TRUNC('month', created_at) AS month,
    COUNT(*) AS total_rfps,
    COUNT(*) FILTER (WHERE status = 'ACCEPTED') AS accepted,
    COUNT(*) FILTER (WHERE status = 'DECLINED') AS declined,
    AVG(score) AS avg_score,
    AVG(EXTRACT(EPOCH FROM (review_timestamp - created_at)) /
FROM rfps
GROUP BY DATE_TRUNC('month', created_at)
ORDER BY month DESC;

-- View: Partner Performance
CREATE VIEW v_partner_performance AS
SELECT
    p.partner_id,
    p.company_name,
    p.total_bids,
    p.total_wins,
    p.win_rate,
    COUNT(b.id) AS current_period_bids,
    COUNT(b.id) FILTER (WHERE b.status = 'ACCEPTED') AS currer
    AVG(b.proposed_fee) AS avg_fee,
    AVG(EXTRACT(EPOCH FROM (b.submitted_at - b.created_at)) /
FROM partners p
LEFT JOIN bids b ON b.partner_id = p.id
WHERE b.created_at >= NOW() - INTERVAL '6 months'
GROUP BY p.id, p.partner_id, p.company_name, p.total_bids, p.t

-- View: Win/Loss Analysis by Client
CREATE VIEW v_client_performance AS
SELECT
    client_name,
    COUNT(*) AS total_rfps,
    COUNT(*) FILTER (WHERE status = 'ACCEPTED') AS bids_submit
    COUNT(b.id) FILTER (WHERE b.status = 'ACCEPTED') AS bids_w
    ROUND(
        COUNT(b.id) FILTER (WHERE b.status = 'ACCEPTED')::NUME
        NULLIF(COUNT(*) FILTER (WHERE status = 'ACCEPTED'), 0)
        2
    ) AS win_rate,
    SUM(b.proposed_fee) FILTER (WHERE b.status = 'ACCEPTED') A
FROM rfps r
LEFT JOIN bids b ON b.rfp_id = r.id
GROUP BY client_name
ORDER BY total_rfps DESC;
```

7. Deployment Architecture

7.1 Infrastructure Components

```
┌──────────────────────────────────────────────────────────────────────────────
│                        LOAD BALANCER (AWS ALB)
│                         SSL Termination
└──────────────────────────┬──────────────────────────────────────────────────
                           │
          ┌────────────────┴────────────────┐
          │                                 │
┌─────────▼─────────┐               ┌────────▼────────┐
│  FRONTEND (S3)    │               │  API GATEWAY    │
│   CloudFront      │               │   (ECS Fargate  │
│   React Build     │               │    3 tasks)     │
└───────────────────┘               └────────┬────────┘
                                             │
                        ┌────────────────────┴────────────────────┐
                        │                                         │
                ┌───────▼──────────┐                   ┌──────────▼──
                │ CELERY WORKERS   │                   │ REDIS CLUSTER
                │ (ECS Fargate     │◄──────────────────┤ (ElastiCache)
                │  5-10 tasks)     │                   │ - Task queue
                └──────────────────┘                   │ - Result back
                         │                             └──────────────
                         │
                ┌────────▼──────────────────────────────────────┐
                │        POSTGRESQL (RDS)                       │
                │        - Primary (Multi-AZ)                   │
                │        - Read Replica (Analytics)             │
                │        - Automated backups                    │
                └───────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────────
│                       S3 BUCKETS
│  - b-bidding-rfps (RFP documents)
│  - b-bidding-contracts (Contract files)
│  - b-bidding-templates (Word/Excel templates)
│  - b-bidding-reports (Generated reports)
└──────────────────────────────────────────────────────────────────────────────

┌──────────────────────────────────────────────────────────────────────────────
│                   MONITORING & OBSERVABILITY
│  - CloudWatch (Logs, Metrics, Alarms)
│  - Sentry (Error tracking)
│  - Datadog (APM, Infrastructure monitoring)
└──────────────────────────────────────────────────────────────────────────────
```

7.2 Docker Compose (Development)

```yaml
# docker-compose.yml
version: '3.8'

services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: b_bidding
      POSTGRES_USER: rufus
      POSTGRES_PASSWORD: rufus_password
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: minio_admin
      MINIO_ROOT_PASSWORD: minio_password
    ports:
      - "9000:9000"
      - "9001:9001"
    volumes:
      - minio_data:/data

  api:
    build:
      context: .
      dockerfile: Dockerfile.api
    environment:
      DATABASE_URL: postgresql://rufus:rufus_password@postgres
      REDIS_URL: redis://redis:6379/0
      S3_ENDPOINT: http://minio:9000
      S3_ACCESS_KEY: minio_admin
      S3_SECRET_KEY: minio_password
      OPENAI_API_KEY: ${OPENAI_API_KEY}
      ANTHROPIC_API_KEY: ${ANTHROPIC_API_KEY}
    ports:
      - "8000:8000"
    depends_on:
      - postgres
      - redis
      - minio
    command: uvicorn b_bidding_api.main:app --host 0.0.0.0 --p
```

```yaml
  celery_worker:
    build:
      context: .
      dockerfile: Dockerfile.api
    environment:
      DATABASE_URL: postgresql://rufus:rufus_password@postgres
      REDIS_URL: redis://redis:6379/0
      S3_ENDPOINT: http://minio:9000
      S3_ACCESS_KEY: minio_admin
      S3_SECRET_KEY: minio_password
      OPENAI_API_KEY: ${OPENAI_API_KEY}
      ANTHROPIC_API_KEY: ${ANTHROPIC_API_KEY}
    depends_on:
      - postgres
      - redis
      - minio
    command: celery -A b_bidding_api.celery_app worker --logle

  celery_beat:
    build:
      context: .
      dockerfile: Dockerfile.api
    environment:
      DATABASE_URL: postgresql://rufus:rufus_password@postgres
      REDIS_URL: redis://redis:6379/0
    depends_on:
      - postgres
      - redis
    command: celery -A b_bidding_api.celery_app beat --logleve

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    ports:
      - "3000:80"
    depends_on:
      - api

volumes:
  postgres_data:
  minio_data:
```

7.3 Kubernetes Deployment (Production)

```yaml
# k8s/api-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: b-bidding-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: b-bidding-api
  template:
    metadata:
      labels:
        app: b-bidding-api
    spec:
      containers:
      - name: api
        image: your-registry/b-bidding-api:latest
        ports:
        - containerPort: 8000
        env:
        - name: DATABASE_URL
          valueFrom:
            secretKeyRef:
              name: b-bidding-secrets
              key: database-url
        - name: REDIS_URL
          valueFrom:
            secretKeyRef:
              name: b-bidding-secrets
              key: redis-url
        resources:
          requests:
            memory: "512Mi"
            cpu: "500m"
          limits:
            memory: "1Gi"
            cpu: "1000m"
        livenessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 30
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /ready
            port: 8000
          initialDelaySeconds: 10
          periodSeconds: 5

---
```

```yaml
# k8s/celery-worker-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: b-bidding-celery-worker
spec:
  replicas: 5
  selector:
    matchLabels:
      app: b-bidding-celery-worker
  template:
    metadata:
      labels:
        app: b-bidding-celery-worker
    spec:
      containers:
      - name: worker
        image: your-registry/b-bidding-api:latest
        command: ["celery", "-A", "b_bidding_api.celery_app",
        env:
        - name: DATABASE_URL
          valueFrom:
            secretKeyRef:
              name: b-bidding-secrets
              key: database-url
        - name: REDIS_URL
          valueFrom:
            secretKeyRef:
              name: b-bidding-secrets
              key: redis-url
        resources:
          requests:
            memory: "1Gi"
            cpu: "1000m"
          limits:
            memory: "2Gi"
            cpu: "2000m"

---
# k8s/zombie-scanner-cronjob.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: zombie-workflow-scanner
spec:
  schedule: "*/5 * * * *"  # Every 5 minutes
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            name: scanner
```

```
  - name: scanner
    image: your-registry/b-bidding-api:latest
    command: ["rufus", "scan-zombies", "--db", "$(DATA
    env:
    - name: DATABASE_URL
      valueFrom:
        secretKeyRef:
          name: b-bidding-secrets
          key: database-url
restartPolicy: OnFailure
```

---

8. Implementation Phases

Phase 1: Foundation (Weeks 1-3)

Goals:

- Set up infrastructure
- Implement core workflows
- Basic UI for RFP upload and workflow tracking

Deliverables:

1. **Backend**:
    - FastAPI application scaffolding
    - PostgreSQL database setup with schema
    - Rufus SDK integration
    - Basic auth (JWT)
    - S3 file storage integration
    - Celery worker setup
2. **Custom Step Types**:
    - `FILE_PROCESSOR` step (unzip, validate, extract metadata)
    - `AI_EXTRACTION` step (OpenAI integration for RFP data extraction)
    - `AI_SCORING` step (basic rule-based scoring)
3. **Workflows**:
    - `RFP_Intake` workflow (complete)
    - `Partner_PQQ` workflow (basic)
4. **Frontend**:
    - Login page
    - RFP upload page
    - Workflow list page
    - Workflow detail page (with live updates via polling)

5. **DevOps**:
    - Docker Compose for local development
    - CI/CD pipeline (GitHub Actions)
    - Basic monitoring (CloudWatch)

Testing:

- Upload sample RFP (small PDF)
- Verify AI extraction accuracy
- Test scoring model with sample data

Phase 2: Sub-Consultant Management (Weeks 4-6)

Goals:

- Complete partner prequalification workflow
- Implement consultant invitation and tracking
- Build partner portal

Deliverables:

1. **Backend**:
    - Partner API endpoints (`/api/v1/partners/`)
    - Bid API endpoints (`/api/v1/bids/`)
    - Email notification service integration (SendGrid)
2. **Custom Step Types**:
    - `EMAIL_NOTIFIER` step
    - Enhanced `AI_SCORING` for PQQ
3. **Workflows**:
    - `Consultant_Invitation` workflow (complete)
    - `Consultant_Reminder` workflow (CRON-based)
4. **Frontend**:
    - Partner list and detail pages (internal)
    - Partner portal (external, partner-facing):
        - Login page
        - Dashboard (active invitations)
        - Bid submission form
        - Past submissions history
    - PQQ form page
5. **Database**:
    - `partners` table fully populated

    - `bids` table with submission tracking

Testing:

- End-to-end test: RFP accepted → consultants invited → bids submitted
- Test CRON reminders (use 1-minute interval for testing)

Phase 3: Contract Review & Analytics (Weeks 7-9)

Goals:

- AI contract analysis with White Book comparison
- Analytics dashboards
- Template generation

Deliverables:

1. **Backend**:
    - Contract API endpoints (`/api/v1/contracts/`)
    - Analytics API endpoints (`/api/v1/analytics/`)
2. **Custom Step Types**:
    - `AI_CONTRACT_REVIEW` step (Anthropic Claude integration)
    - `TEMPLATE_GENERATOR` step (python-docx for Word templates)
3. **Workflows**:
    - `Contract_Review` workflow (complete)
4. **Frontend**:
    - Contract upload page
    - Contract analysis results page (clause flagging, risk summary)
    - Analytics dashboard:
        - KPI cards (total RFPs, win rate, pipeline value)
        - Win/loss trend chart
        - Top clients table
        - Partner performance table
    - Fee analysis page
5. **Database**:
    - `contracts` table
    - `analytics_metrics` table
    - Database views for analytics

Testing:

- Upload sample contract PDF
- Verify clause extraction and risk flagging

- Validate analytics calculations

Phase 4: Advanced Features & Polish (Weeks 10-12)

Goals:

- Document automation (technical/commercial proposals)
- Advanced analytics with drill-through
- **OPTIONAL**: RLM (Recursive Language Models) pilot for complex multi-document analysis
- Performance optimization
- Production hardening

Deliverables:

1. **Backend**:
    - Zombie workflow scanner (production deployment)
    - WebSocket support for real-time updates
    - Rate limiting and API throttling
    - Enhanced error handling and retry logic
    - Performance optimizations (caching, query optimization)
2. **Custom Step Types**:
    - `TEMPLATE_GENERATOR` with Excel support
    - Enhanced `AI_EXTRACTION` with OCR for scanned PDFs
    - **OPTIONAL**: `RLM_QUERY` step for multi-document recursive analysis (see Section 2.4)
        - A/B test on 20 high-value contracts (>$1M)
        - Decision criteria: Proceed if 15%+ accuracy gain OR 25%+ cost savings
3. **Workflows**:
    - `Bid_Submission_Workflow` (technical + commercial proposal generation)
    - Workflow versioning and migration strategy
4. **Frontend**:
    - Real-time WebSocket updates (replace polling)
    - Advanced analytics:
        - Drill-through from charts to underlying data
        - Custom date range filters
        - Export to CSV/Excel
    - Document preview (PDF inline viewing)
    - Bulk operations (multi-select RFPs, batch actions)
    - Mobile-responsive design improvements

5. **DevOps**:
    - Kubernetes deployment (EKS)
    - Production monitoring (Datadog, Sentry)
    - Automated backups and disaster recovery
    - Load testing (target: 100 concurrent workflows)
    - Security audit (OWASP top 10)
6. **Documentation**:
    - API documentation (Swagger/OpenAPI)
    - User guide (internal users)
    - Partner portal guide
    - Admin guide (system configuration)

Testing:

- Load testing (simulate 1000 RFPs/day)
- Security testing (penetration testing)
- UAT with Bronwyn and team

Phase 5: Go-Live & Support (Weeks 13-14)

Goals:

- Production deployment
- User training
- Monitoring and support

Deliverables:

1. **Deployment**:
    - Production environment setup (AWS)
    - Data migration (if migrating from existing system)
    - DNS configuration
2. **Training**:
    - Training sessions for internal users (2 hours)
    - Training sessions for partner consultants (1 hour)
    - Video tutorials (screen recordings)
3. **Support**:
    - On-call support for first 2 weeks
    - Bug tracking (Jira/Linear)
    - Feedback collection and prioritization
4. **Monitoring**:

- Daily system health checks
- Weekly analytics reviews (win rate, avg response time)
- Monthly performance optimization

---

9. Risk Mitigation

9.1 Technical Risks

Risk	Impact	Probability	Mitigation Strategy
AI extraction errors (incorrect GFA, missing deadlines)	High	Medium	- Use structured prompts with examples - Implement validation rules - Always require human review - Track extraction accuracy metrics
Large file processing timeouts (10 GB RFP zips)	High	Medium	- Stream files to S3 first - Process in chunks (1000 files/batch) - Use Celery with increased timeout - Progress tracking for users
Celery worker crashes (zombie workflows)	Medium	Low	- Use HeartbeatManager (built-in) - Run ZombieScanner as CronJob - Alert on stale workflows
Database performance (concurrent writes)	Medium	Low	- Use PostgreSQL with proper indexing - Connection pooling (min=10, max=50) - Read replicas for analytics
API rate limits (OpenAI/Anthropic)	Medium	Medium	- Implement exponential backoff

			- Queue requests via Celery - Monitor usage and scale
S3 storage costs (large RFP files)	Low	Medium	- Use S3 Intelligent-Tiering - Archive old RFPs to Glacier - Implement retention policies

9.2 Business Risks

Risk	Impact	Probability	Mitigation Strategy
Poor AI recommendation accuracy	High	Medium	- Start with human-in-the-loop - Track override reasons - Continuously refine scoring model - Set recommendation as "suggestion" not "decision"
Low partner adoption (portal usage)	Medium	High	- Make portal simple and valuable - Highlight time savings - Offer training and support - Collect feedback early
Consultant resistance (prefer email)	Medium	Medium	- Support both email and portal - Gradually migrate to portal - Show benefits (auto-reminders, history)
Data privacy concerns (client confidentiality)	High	Low	- Encrypt data at rest and in transit - Role-based access control - Audit logs for all access - Compliance review (GDPR if applicable)

9.3 Contingency Plans

Plan A: AI Extraction Fails

- Fallback to manual data entry form
- Pre-populate with OCR results (Tesseract)
- Flag for human verification

Plan B: Celery Workers Overloaded

- Auto-scale workers (Kubernetes HPA)
- Queue priority (urgent RFPs first)
- Offload heavy tasks to batch jobs

Plan C: Database Outage

- RDS Multi-AZ automatic failover
- Read replicas for queries
- Rufus workflows resume after recovery (state persisted)

---

10. Success Metrics

10.1 System Performance Metrics

Metric	Target	Measurement
RFP Processing Time	< 30 minutes (intake to human review)	Workflow audit logs
AI Extraction Accuracy	> 90% correct fields	Human override tracking
Scoring Model Accuracy	> 80% agreement with human decision	Override rate
Consultant Response Rate	> 70% submit bids on time	Bid submission tracking
API Response Time (p95)	< 500ms	CloudWatch metrics
Workflow Throughput	100 RFPs/day	Persistence metrics
System Uptime	> 99.5%	Monitoring alerts

10.2 Business Metrics

Metric	Baseline	Target (3 months)	Target (6 months)
Time to Bid/No-Bid Decision	2-3 days	4-8 hours	2-4 hours
Consultant Coordination Time	5-10 hours/RFP	2-3 hours/RFP	< 1 hour/RFP
Missed RFPs	5-10%	< 2%	< 1%
Win Rate	Baseline (track)	+5%	+10%
Consultant Satisfaction	Baseline (survey)	7/10	8/10
Number of Active Partners	Baseline	+20%	+50%

10.3 Adoption Metrics

Metric	Target (Month 1)	Target (Month 3)	Target (Month 6)
Portal Logins (Partners)	30% of partners	60% of partners	80% of partners
RFPs Processed via System	50%	80%	95%
User Satisfaction (Internal)	7/10	8/10	9/10
Support Tickets	< 10/week	< 5/week	< 2/week

11. Next Steps

Immediate Actions (This Week)

1. **Technical Setup**:
   - [ ] Create project repositories (backend, frontend, rufus-bidding-steps)
   - [ ] Set up development environment (Docker Compose)
   - [ ] Initialize PostgreSQL database with schema
   - [ ] Configure S3/MinIO for file storage
2. **Requirements Gathering**:

- [ ] **Request from Bronwyn**:
    - Sample RFPs (1 small, 1 large, 1 with ZIP structure)
    - Bid/no-bid scoring sheet (Excel or PDF)
    - Historical bid tracker (wins/losses, reasons)
    - Technical and commercial proposal templates (Word/Excel)
    - "White Book" reference document (PDF)
    - Example contracts with risky clauses
    - Screen recording of current process (15-20 min)
    - Replit prototype link (if available)
- [ ] Schedule follow-up meeting to review materials

3. **Team Alignment**:
    - [ ] Assign roles (backend dev, frontend dev, DevOps, QA)
    - [ ] Set up project tracking (Jira/Linear/GitHub Projects)
    - [ ] Schedule weekly sprint planning
4. **Prototyping**:
    - [ ] Build minimal RFP upload → AI extraction → display results
    - [ ] Validate OpenAI API integration for extraction
    - [ ] Test Anthropic Claude for contract review

Follow-Up Meeting Agenda (Next Week)

1. Review materials from Bronwyn
2. Demo prototype (RFP upload + extraction)
3. Validate scoring model rules
4. Discuss partner prequalification criteria
5. Prioritize Phase 1 features
6. Confirm timeline and resources

Appendix A: Technology Justifications

Why Rufus SDK?

1. **Workflow-Native**: Built for complex, multi-step workflows with human-in-the-loop
2. **Production-Ready**: Saga pattern, zombie recovery, definition snapshots
3. **Extensible**: Custom step types via marketplace pattern
4. **Portable**: Works with SQLite (dev) and PostgreSQL (prod)
5. **Observability**: Built-in audit logs, metrics, and event hooks
6. **Performance**: uvloop, orjson, connection pooling (50-100% throughput gains)

Why FastAPI?

1. **Async-First**: Matches Rufus SDK's async architecture
2. **High Performance**: On par with Node.js and Go
3. **Type Safety**: Pydantic models align with Rufus state models
4. **Auto-Documentation**: OpenAPI/Swagger out-of-the-box
5. **Ecosystem**: Rich ecosystem for auth, validation, testing

Why Celery?

1. **Distributed Execution**: Scale workers independently
2. **Mature**: Battle-tested in production (10+ years)
3. **Monitoring**: Flower UI for task monitoring
4. **Flexibility**: Priority queues, retries, rate limiting
5. **Rufus Integration**: ExecutionProvider already designed for Celery

Why React + TypeScript?

1. **Type Safety**: Catch errors at compile time
2. **Large Ecosystem**: UI libraries (shadcn/ui), charting (Recharts)
3. **Developer Experience**: Fast refresh, tooling (Vite)
4. **Industry Standard**: Easy to find developers
5. **Scalability**: Component-based architecture

Appendix B: Sample API Request/Response

Upload RFP and Start Workflow:

```
curl -X POST "http://localhost:8000/api/v1/rfps/upload" \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXV(
  -F "file=@/path/to/rfp_package.zip" \
  -F 'metadata={"source":"email","priority":"high"}'
```

Response:

```
{
  "rfp_id": "rfp_20260211_a8f2c4e1",
  "workflow_id": "550e8400-e29b-41d4-a716-446655440000",
  "status": "ACTIVE",
  "current_step": "Process_Files",
  "s3_key": "rfps/2026-02-11/rfp_20260211_a8f2c4e1.zip",
  "created_at": "2026-02-11T10:05:23.123Z",
  "estimated_completion": "2026-02-11T10:35:00.000Z"
}
```

Get Workflow Status:

```
curl -X GET "http://localhost:8000/api/v1/workflows/550e8400-e
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC
```

Response:

```
{
  "workflow_id": "550e8400-e29b-41d4-a716-446655440000",
  "workflow_type": "RFP_Intake",
  "status": "WAITING_HUMAN",
  "current_step": "Human_Review",
  "state": {
    "rfp_id": "rfp_20260211_a8f2c4e1",
    "project_name": "Dubai Marina Mixed-Use Tower",
    "client_name": "Emaar Properties",
    "location": "Dubai, UAE",
    "gfa_sqm": 85000,
    "submission_deadline": "2026-03-15T12:00:00Z",
    "score": 85,
    "recommendation": "A",
    "scoring_reasons": [
      "Project size matches target range (50k-100k sqm): +25 p
      "Prime location (Dubai Marina): +30 points",
      "Existing client with good history: +20 points",
      "AI strategic analysis: high-value opportunity: +10 poir
    ],
    "approved": null
  },
  "steps": [
    {
      "name": "Store_RFP_Package",
      "status": "COMPLETED",
      "completed_at": "2026-02-11T10:05:30Z"
    },
    {
      "name": "Process_Files",
      "status": "COMPLETED",
      "completed_at": "2026-02-11T10:08:45Z"
    },
    {
      "name": "Extract_RFP_Data",
      "status": "COMPLETED",
      "completed_at": "2026-02-11T10:12:20Z"
    },
    {
      "name": "Score_RFP",
      "status": "COMPLETED",
      "completed_at": "2026-02-11T10:12:35Z"
    },
    {
      "name": "Human_Review",
      "status": "ACTIVE",
      "started_at": "2026-02-11T10:12:36Z"
    }
  ],
  "created_at": "2026-02-11T10:05:23Z",
  "updated_at": "2026-02-11T10:12:36Z"
}
```

Conclusion

This implementation plan outlines a comprehensive, production-ready AI pilot for the B Bidding Master system. By leveraging **Rufus SDK's workflow orchestration**, **FastAPI's high performance**, and **custom AI-powered step types**, we can deliver:

1. **60-80% reduction** in manual RFP processing time
2. **Automated sub-consultant coordination** with real-time tracking
3. **AI-powered contract risk analysis** for faster decisions
4. **Comprehensive analytics** for continuous improvement

The phased approach ensures incremental delivery of value, with Phase 1 (Foundation) delivering core functionality in 3 weeks. The architecture is designed for **scalability** (100+ RFPs/day), **reliability** (zombie recovery, Saga pattern), and **extensibility** (marketplace-based custom steps).

Recommendation: Proceed with Phase 1 implementation immediately. The technical stack (Rufus SDK + FastAPI + React) is well-suited for this use case and represents a proven, production-ready AI architecture.

Document prepared by: Claude Sonnet 4.5 **For**: B Bidding AI Pilot Project **Date**: 2026-02-11